

Legacy Banking Modernisation Platform

Proving the rewrite behaves exactly like the original.

Legacy migration fails on verification, not translation. We make behavioural equivalence something a bank can prove, then check that answer against the regulator.



Team StudyEdge Odisha University of Technology and Research, Bhubaneswar

Tamanna Panda · Adyasha Das · Soumyajit Sarkar · Swayam Subhankar Sahoo

India’s banks run on code nobody can safely change

92

of the top 100 banks worldwide run on IBM mainframes

40%+

of banking systems still run COBOL written decades ago

\$1bn

over 5–10 years to upgrade Indian core banking systems (BCG)

0

specifications state what those programs should do

When a bank guesses wrong, it is a supervisory event, not a bug report

TSB, United Kingdom — 2018

Failed at cutover. £48.65m FCA/PRA fine, £32.7m customer redress; normal service took eight months to return.

HDFC Bank, India — 2020

After repeated outages, RBI froze every Digital 2.0 launch and all new cards. Restrictions ran into 2022.

Why the backlog is decades long

One paisa across millions of accounts is a reportable event. Not touching the system is the rational choice.

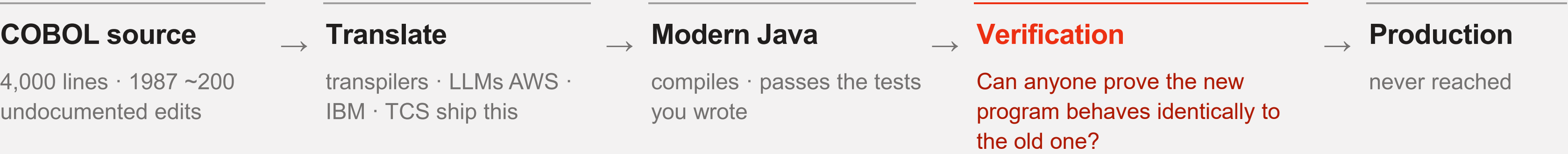
1987 core interest engine written

2018 TSB — £400m failed cutover

2020 HDFC — RBI freezes the roadmap

2026 migration still deferred

Translation is solved. Proof is not.



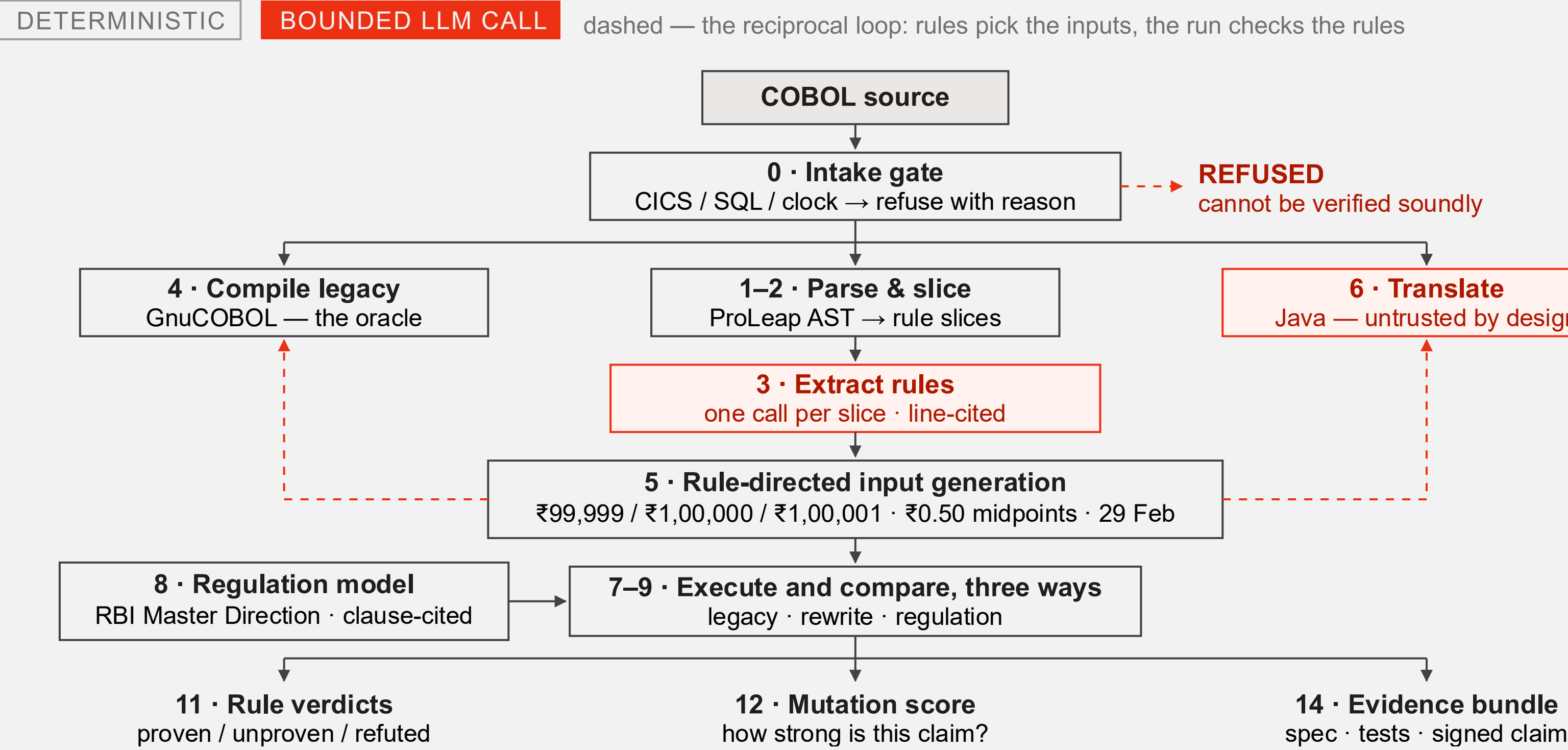
No specification was ever written — the rules live only in the code. AI explanations look excellent; a meaningful share is wrong:

62% precision of the best published COBOL rule extractor A-COBREX, ICSE 2025	91.2% of business rules successfully extracted by a 2026 system AgentModernize, arXiv 2605.17535	<20% behavioural equivalence reached by that same system baselines scored 0.0%
---	---	--

Extraction works. Behavioural equivalence does not follow from it. **Verification has to be the product.**

IBM Research, on IBM’s own product: “the resulting code cannot be trusted to correctly translate the original code” (arXiv:2504.10548). If the vendor needs validation, so does the bank.

The pipeline, end to end



What one finding looks like

The run

input: ₹1,00,000.00 · 12 days · 3.5%

₹115

vs

₹116

legacy COBOL

generated Java

Δ = ₹1 per account, per accrual — across millions of accounts.

Traced to one word

Delta debugging shrinks 73 failing inputs to this one. Coverage maps it to rule R-07 — line 214:

```
COMPUTE WS-INT ROUNDED
```

The Java rounds **HALF_UP**; the COBOL truncates at the PIC boundary. No crash, no exception — just a different number.

The regulator has ruled

“Rounded off to the nearest rupee... fifty paise and above rounds to the next higher rupee.”

RBI Master Direction — Interest Rate on Deposits, clause 4(f) · Daily product basis, clause 6(a) · Uniform rate to ₹1 lakh, 6(a)(1)–(2).

And the report it ships inside

31	11	5	94%
rules proven	unproven — named	refuted — our AI caught	mutation kill rate

Three things nobody else does

The differential mechanism itself is prior art. IBM, AWS, Mechanical Orchard and a paper published five days ago all ship a version of it, and we say so. These three are ours.

01

We check the legacy against the regulator

Every tool treats the legacy as correct by definition. A program written in 1987 can breach a Direction issued decades later, so we add a third reference.

L = R = Reg	verified compliant
L ≠ R	translation defect
L = R ≠ Reg	inherited breach
L ≠ R = Reg	silent correction

Only the third row needs a regulator. No other tool can see it.

02

A result for every rule, not every test case

Executing a line is not testing the rule it encodes. A rule counts as confirmed only when all three checks hold.

- 1 Reached — the rule’s lines ran
- 2 Discriminating — the boundary changed the output
- 3 Agreed — the rewrite matched every time

Check 2 is the one that catches our own extraction inventing a rule that is not in the program.

03

We test our own tests

“Everything passed” proves nothing unless the inputs could have caught a failure. So we break the legacy on purpose.

Inject a deliberate defect 100000 → 100001 · > → >= · delete ROUNDED · 360 → 365

Re-run the same input corpus

Caught the claim stands

Missed we do not claim it

The equivalence claim ships with its own strength score.

Checked against Locksmith (arXiv 2607.28271), AgentModernize (arXiv 2605.17535) and the shipping vendor tools. None reports per-rule results, scores its own corpus, or consults a regulator.

The spend is already committed. Assurance is the gate.

~\$1bn

Indian lenders must invest over 5–10 years to upgrade legacy core banking
Boston Consulting Group, 2024

92/100

of the world’s largest banks run core operations on IBM mainframes
widely reported industry figure

>40%

of banking systems still run COBOL in production today
Pragmatic Coders, 2025

40,000+

mobile & internet banking complaints, FY22–23
RBI ombudsman data

Every rupee of it is gated by one step

Budgets are approved. What stalls them is the step between “the code exists” and “we run it on real money” — no product category yet; absorbed as consulting hours and schedule risk.

The category is forming around generation

AWS Transform for mainframe — GA May 2025; ADP cut extraction time 80%. **IBM watsonx Code Assistant for Z** — ships a validation assistant; IBM’s own papers say the output cannot be trusted unchecked. **Mechanical Orchard, Imogen** — a generate-then-validate loop.

All of them generate. None of them sells the proof as the product.

Indian banks spend up to 5% of revenue on IT against 7–9% for global peers — the gap compounds: less budget → fewer modernisation attempts → more undocumented logic.

The plan on a page: why, how, what

WHY

explain the problem

Problem & business scenario — 92 of the top 100 banks run on mainframes; over 40% of banking systems still run COBOL. Indian lenders face ~\$1bn of core upgrades over 5–10 years (BCG). The rules were never documented anywhere except the code.

Problem scope — COBOL, pure-computation programs, retail interest accrual. CICS/JCL, database-coupled and non-deterministic programs are excluded — detected and refused at intake, not silently mishandled.

Target users & stakeholders — CIO and core-banking head, risk and internal audit, modernisation SIs, the supervisor. All can generate the new code. None can prove it is safe.

HOW

explain the solve

Solution overview — intake gate → parse and slice → bounded LLM rule extraction with line citations → rule-directed inputs → execute three ways → diff. Outputs: rule verdicts, mutation score, compliance findings, evidence bundle.

Technical details — GnuCOBOL (19 dialects, 9,700/9,748 NIST), ProLeap ANTLR4 parser, program slicing, gcov coverage, delta debugging.

Innovation — the legacy binary is its own oracle; the reciprocal loop lets execution judge the extracted rules; the RBI Direction checks the legacy against the regulator too.

Why these technologies — free, NIST-validated, runs on a laptop with no mainframe. Deterministic orchestration at up to 3.5× lower token cost.

WHAT

value proposition

A specification that never existed — plain-language rules, each traced to the exact lines that produce it.

A permanent regression suite — thousands of input–output pairs, reusable on every future change.

An equivalence claim someone can sign — evidence-backed, coverage stated honestly, per rule.

Compliance findings — the legacy checked against RBI’s written arithmetic, not only against itself.

Value even if the migration is cancelled — the spec and the suite are retained. The bank gets documentation for a system it could not touch.

Financials & timelines

INVESTMENTS

what it takes, and what it costs

Licence cost: nil — GnuCOBOL (GPL/LGPL), ProLeap (MIT), gcov, NIST CCVS85 and AWS CardDemo are all open. No mainframe, no vendor contract.

Prototype — 4 engineers × 12 weeks, on commodity laptops.

Only variable cost is inference — one bounded LLM call per slice; deterministic orchestration measured at up to 3.5× lower token cost than an agentic loop.

Bank pilot — 2 engineers plus SME time from the bank, one program family, about 6 months.

RETURNS

quantified — and the cost of not solving

What it unlocks — the ~\$1bn of Indian core-banking upgrade spend is gated on assurance. Comparable automation (ADP via AWS Transform) cut rule-extraction time 80% and manual effort over 90%.

What if I don't solve — TSB, 2018 — 1.9m of 5.2m customers hit. £48.65m FCA/PRA fine, £32.7m redress, ~£400m total. The CEO resigned; the CIO was personally fined £81,620. The finding was about assurance — not programming.

And in India — RBI halted HDFC's Digital 2.0 launches and new card issuance in Dec 2020; restrictions ran into 2022. An IT failure here freezes the product roadmap.

TIMELINES

time to realise the benefit

Weeks 1–2 — harness runs two programs on an input list and diffs. Already a working demo.

Weeks 3–4 — input generation from PIC clauses plus hand-written boundaries.

Weeks 5–7 — parse, slice, extract rules with line citations.

Weeks 8–10 — the reciprocal loop and per-rule coverage verdicts. This is the contribution.

Weeks 11–12 — delta debugging to minimal counterexamples; evidence bundle generated.

Months 4–9 — bank pilot on one program family, inside the bank's own compiler environment.

Sources

PEER-REVIEWED AND PREPRINTS

Hans, S. et al. (2025). Automated Testing of COBOL to Java Transformation. arXiv:2504.10548.

Pan, R. et al. (2024). Lost in Translation: A Study of Bugs Introduced by LLMs while Translating Code. ICSE 2024, arXiv:2308.03109.

Ahmed, S. N. & Galib, M. (2026). AgentModernize: Preserving Business Logic in Legacy Modernization. arXiv:2605.17535.

Ferenczi, A. et al. (2026). Agentic Method for Deterministic Validation of Legacy Code Migration. arXiv:2607.28271.

Shah, S. et al. (2025). A-COBREX: Identifying Business Rules in COBOL Programs. ICSE 2025, Demonstrations.

B S, C. & Chimalakonda, S. (2025). LLM vs Rule-Based: the COBRAIN Tool. EASE 2025, doi:10.1145/3756681.3756982.

Lwin, N. O. & Kumar, R. (2026). Deterministic vs LLM-Controlled Orchestration for COBOL-to-Python Modernization. arXiv:2605.09894.

Barr, E. T. et al. (2015). The Oracle Problem in Software Testing: A Survey. IEEE TSE 41(5), 507–525.

REGULATORY AND SUPERVISORY

Reserve Bank of India (2016, upd. 2024). Master Direction — Interest Rate on Deposits. Clauses 4(f), 6(a), 6(a)(1)–(2).

Reserve Bank of India (2023). Master Direction on IT Governance, Risk, Controls and Assurance Practices.

Financial Conduct Authority (2022). TSB fined £48.65m for operational resilience failings.

INDUSTRY AND MARKET

Boston Consulting Group (2024). Cloud-based Core Transformations, via Business Standard.

IBM. watsonx Code Assistant for Z — Validation Assistant documentation.

Amazon Web Services (2025). AWS Transform for mainframe, generally available.

Mechanical Orchard. Imogen platform — generate-validate loop.

TOOLS AND STANDARDS

GnuCOBOL Project. GnuCOBOL compiler, GPL/LGPL. NIST COBOL-85: 9,700 of 9,748.

Wolf, U. ProLeap COBOL Parser, MIT licence.

NIST (1992). CCVS85 validation system.